TAREA INTRODUCCION A LA INGENIERIA DE SOFTWARE

1. **Una función en programación es un bloque de código reutilizable que realiza una tarea especifica. En el lenguaje C, una función se define especificando el tipo de dato que va a devolver, un nombre único, los parámetros de entrada entre paréntesis y el bloque de códigos entre llaves.**

tipo_de_retorno nombre_de_la_funcion (tipo_parametro1 param1, tipo_parametro2 param2){

//cuerpo de la función: instrucciones a ejecutar

return resultado; //opcional, según el tipo de retorno

}

2. **Cuando se invoca o es llamada una función, la ejecución de el programa principal se pausa temporalmente, se reserva espacio en memoria (pila o stack) para sus variables locales y parámetros, se ejecuta el código que contiene la función de arriba abajo, y finalmente se regresa el control a la línea de código donde fue llamada (retornando un valor si corresponde).**
3. **Float ventas[3][5];**
   // 0 con tipo int dependiendo de si manejas enteros:
   // int ventas[3][5];
4. **#include <stdio.h>**
   #include<ctype.h>

```
Int main() {
   char cadena[100];
   printf("Introduce una cadena: ");
   fgets(cadena, sizeof(cadena), stdin);

   for (int i=0; cadena[i] != '\0'; i++){
        char c= tolower(cadena[i]);
        if(c== 'a' || c == 'i' || c == 'o' || c == 'u'){
           cadena[i] = toupper(cadena[i];
         }
}
```

printf("Cadena modificada: %s", cadena);
return 0;
}

5. **Un apuntador es una variable especial cuyo valor es la dirección de memoria de otra variable en lugar de un dato directo. Y se declara usando el asterisco * antes del nombre de la variable.**
   Int *ptr;
6. **Pues printf("x=%d",x): Imprime el valor del apuntador x, que corresponde a la dirección de memoria hexadecimal (convertida a decimal por %d) de la variable y.**
   Y printf("x=%d", *x): Imprime el valor almacenado en la dirección apuntada por x (es decir, usa el operador de indirección *), lo que mostrara el numero 7.
7. **Antes de ejecutar la línea 7:**
   i = ???? (100)
   ptrToi = ???? (104)
   ptrToPtrToi = ???? (108)
   Tras ejecutar la línea 8 (prtToPtrToi = &ptrToi;):
   i = ???? (100)
   ptrToi = (104)
   ptrToPtrToi = 104 (108) (guarda la dirección de ptrToi)
   Tras ejecutar la línea 10 (ptrToi = &i;):
   i = ???? (100)
   ptrToi = 100 (104) (guardia la dirección de i)
   ptrToPtrToi = 104 (108)
   Antes de ejecutar la línea 11 (i = 10;):
   i = ???? (100)
   ptrToi = 100 (104)
   ptrToPtrToi = 104 (108)
   Tras ejecutar la línea 12 (*ptrToi = 20;):
   i = 20 (100) (modifica el valor de i indirectamente)
   ptrToi = 100 (104)
   ptrToPtrToi = 104 (108)
   Tras ejecutar la línea 10 de la segunda sección (o equivalente a *ptrToPtrToi = 30;):
   i = 30 (100) (modificar el valor de i a través de doble apuntador)
   ptrToi = 100 (104)
   ptrToPtrToi = 104 (108)

8. **malloc (Memory Allocation): Reserva un bloque de memoria del tamaño especificado en bytes, pero no la inicializa, por lo que contiene "basura" (valores aleatorios previos en esa memoria).**
   calloc (Contiguous Allocation): Reserva memoria para un numero determinado de elementos de cierto tamaño y, a diferencia de malloc, inicializa todos los bytes en ceros (0).
9. **#include <stdio.h>**
   #include <stdlib.h>

```
typedef struct {
  float x;
  float y;
  float z;
} cord3D;
int main() {
  int n;
  printf("Introduce el numero de puntos: ");
  scanf("%d", &n);
  // Declaración dinámica con apuntador
  cord3D *puntos = (cord3D *)malloc(n * sizeof(cord3D));
  if (puntos == NULL) {
    printf("Error de asignación de memoria\n");
    return 1;
  }
  // Leer los puntos
  for (int i = 0; i < n; i++) {
    printf("Punto %d (x y z): ", i + 1);
    scanf("%f %f %f", &puntos[i].x, &puntos[i].y, &puntos[i].z);
  }
  // Imprimir los puntos ingresados
  printf("\n--- Puntos Ingresados ---\n");
  for (int i = 0; i < n; i++) {
    printf("Punto %d: (%.2f, %.2f, %.2f)\n", i + 1, puntos[i].x, puntos[i].y,
puntos[i].z);
  }
  free(puntos); // Liberar memoria
  return 0;
}
```

```
10. void guardarPuntos(cord3D *puntos, int n, int esBinario) {
      if (esBinario) {
        FILE *archivo = fopen("puntos.bin", "wb");
        if (archivo != NULL) {
          fwrite(puntos, sizeof(cord3D), n, archivo);
          fclose(archivo);
          printf("Puntos guardados en binario exitosamente.\n");
        }
      } else {
        FILE *archivo = fopen("puntos.txt", "w");
        if (archivo != NULL) {
          for (int i = 0; i < n; i++) {
            fprintf(archivo, "%.2f %.2f %.2f\n", puntos[i].x, puntos[i].y,
    puntos[i].z);
          }
          fclose(archivo);
          printf("Puntos guardados en texto exitosamente.\n");
        }
      }
    }
```